

University of Frontier Technology, Bangladesh

Department of Cyber Security Engineering (CySE)

SEC 204 | Threat Modelling and Security Monitoring Sessional

Lab Manual

Week-4-Lab-Windows-Log-Analysis-with-PowerShell.md

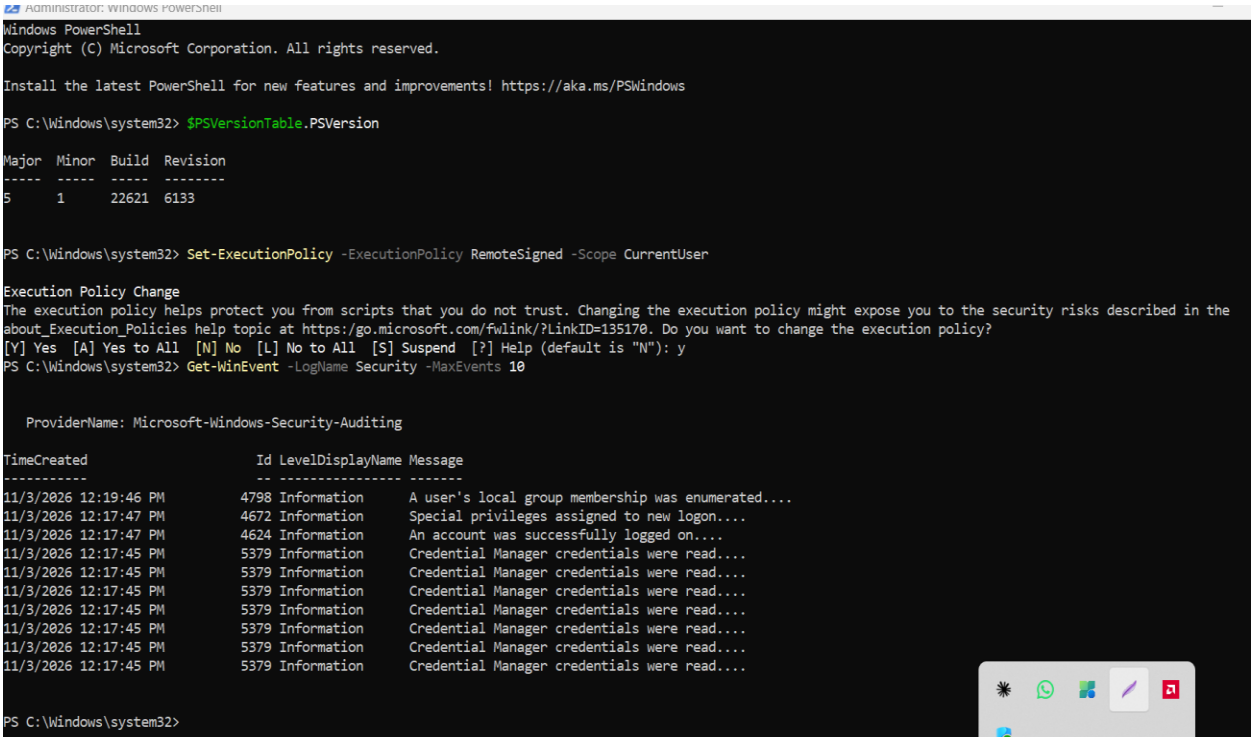
Field	Details
Course Title	Threat Modelling and Security Monitoring Sessional
Course Code	SEC 204
Level & Term	Level 2, Term 1
Instructor	Md Masud Rana, Lecturer, Dept. of CySE, UFTB
Submission Date	03 Nov 2026

Members

Student Name	Student ID	Email
Shanta Islam	2404003	2404003@uftb.ac.bd

Exercise 1: View Recent Security Events

Get-WinEvent -LogName Security -MaxEvents 10



Explanation:

- Get-WinEvent retrieves events from the Windows event logs.

- -LogName Security specifies the Security log.

- -MaxEvents 10 limits the output to the 10 most recent events.

Observation:

The output displays the 10 most recent Security log events. It includes Event ID 4798 (a user's local group membership was enumerated), Event ID 4672 (special privileges assigned to new logon), Event ID 4624 (an account was successfully logged on), and several Event ID 5379 entries (Credential Manager credentials were read).

Exercise 2: Filter by Event ID (Successful Logons, Event ID 4624)

Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEvents 20

```
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEvents 20


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 12:17:47 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:17:42 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:13:02 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:05:05 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:03:54 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:03:02 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:56:27 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:51:53 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:47:41 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:37:08 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:37:08 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:36:57 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:36:53 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:36:53 AM         4624 Information      An account was successfully logged on....
11/3/2026 11:36:53 AM         4624 Information      An account was successfully logged on....
11/3/2026 10:26:29 AM         4624 Information      An account was successfully logged on....
11/3/2026 10:26:28 AM         4624 Information      An account was successfully logged on....
11/3/2026 10:26:28 AM         4624 Information      An account was successfully logged on....
11/3/2026 10:26:23 AM         4624 Information      An account was successfully logged on....
11/3/2026 10:14:35 AM         4624 Information      An account was successfully logged on....


PS C:\Windows\system32>
```

Explanation:

- Get-WinEvent retrieves events from the Windows event logs.

- -FilterHashtable is an efficient filtering method that filters events at the source instead of retrieving all events first.

- @{LogName='Security'; ID=4624} is a hashtable containing the filter criteria: the Security log and Event ID 4624 (successful logon).

- -MaxEvents 20 limits the output to the 20 most recent matching events.

Observation:

The output displays the 20 most recent successful logon events (Event ID 4624) from the Security log. All entries show the level "Information" and the message "An account was successfully logged on." The events were recorded on 11/3/2026 between 10:14:35 AM and 12:17:47 PM. Several events occurred within the same second (for example, at 11:36:53 AM

and 10:26:28 AM), which is normal because Windows creates multiple logon sessions when the system starts services or when a user signs in. No failed logon attempts are shown here, since this query filters only for Event ID 4624.

Exercise 3: Filter by Time Range

Command 1 (last 24 hours):

$StartTime = (Get-Date).AddDays(-1)

Get-WinEvent -FilterHashtable @{

LogName='Security'

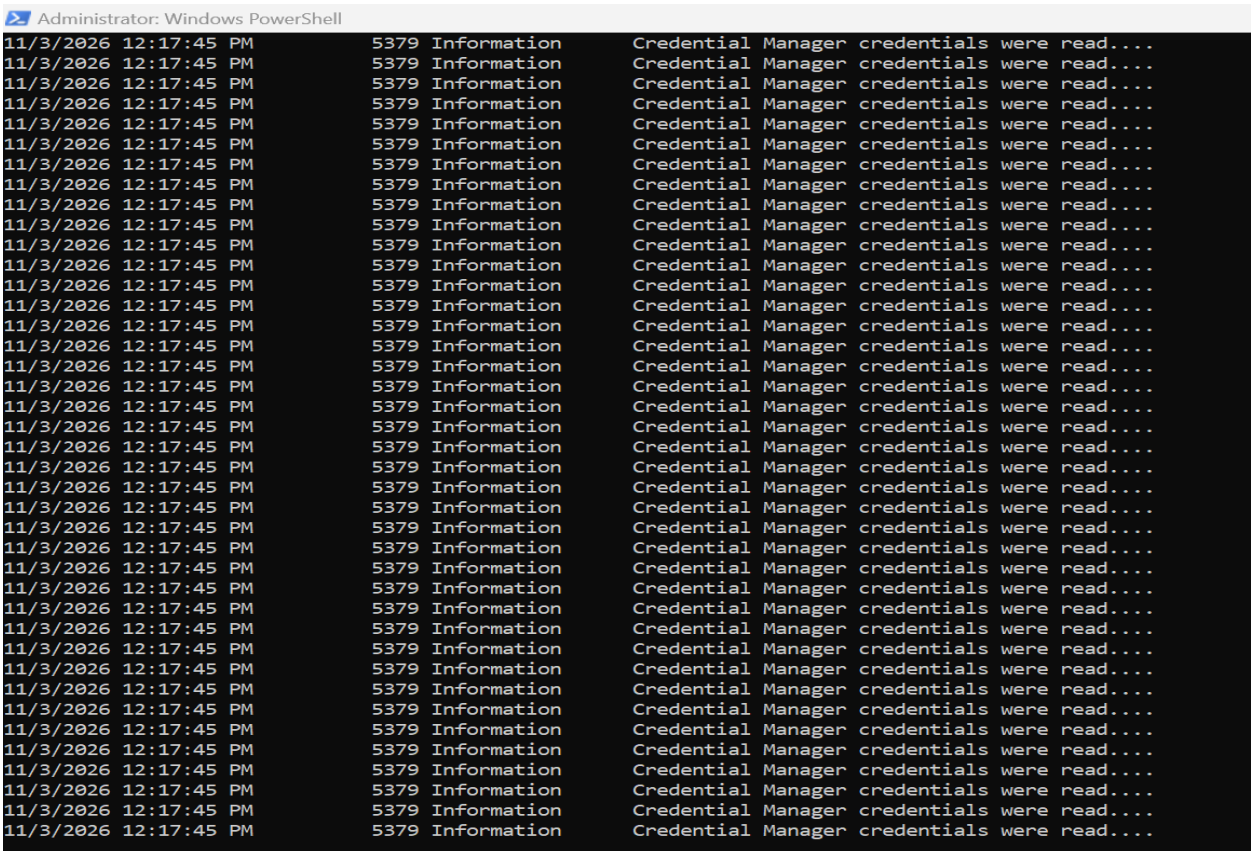
StartTime=$StartTime

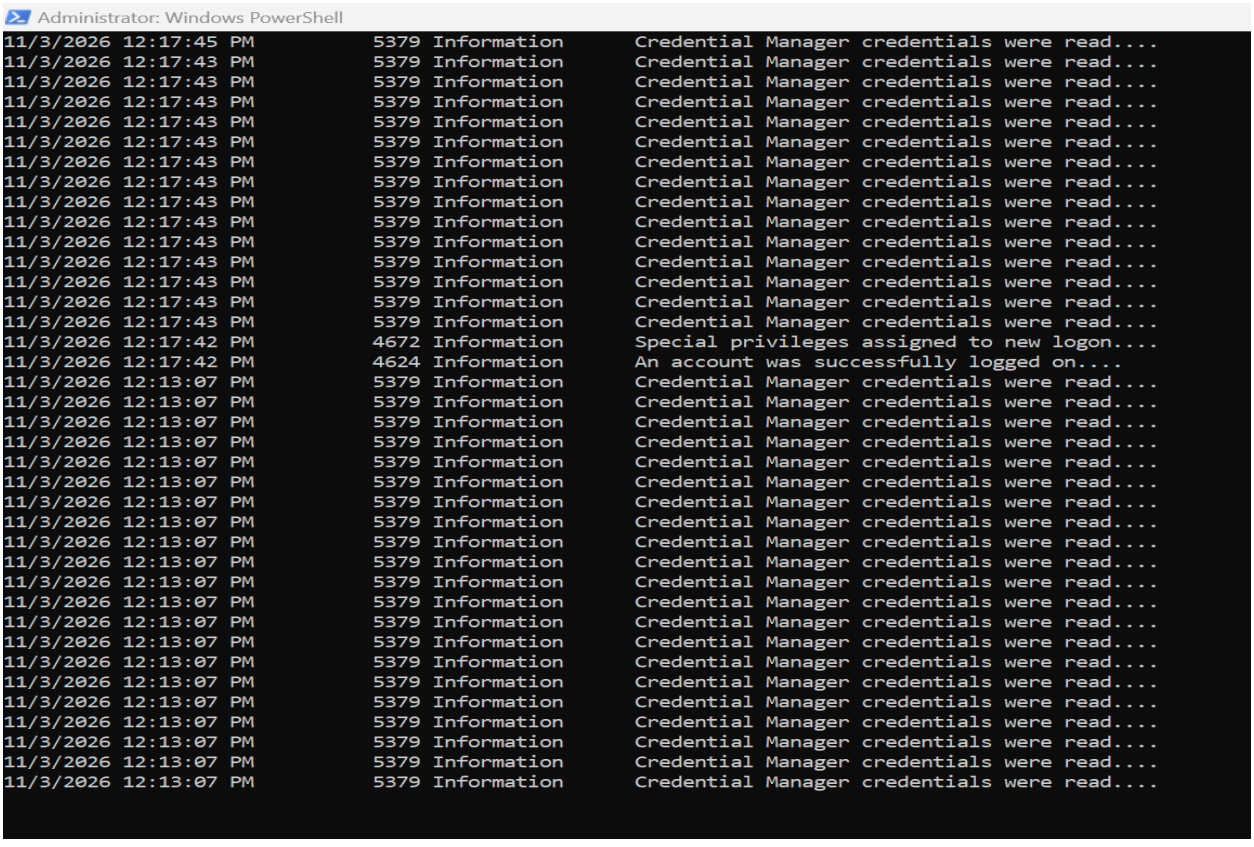
} -MaxEvents 100

```
PS C:\Windows\system32> $StartTime = (Get-Date).AddDays(-1)
>> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     StartTime=$StartTime
>> } -MaxEvents 100


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 12:37:04 PM         4672 Information      Special privileges assigned to new logon....
11/3/2026 12:37:04 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:33:07 PM         4798 Information      A user's local group membership was enumerated....
11/3/2026 12:19:46 PM         4798 Information      A user's local group membership was enumerated....
11/3/2026 12:17:47 PM         4672 Information      Special privileges assigned to new logon....
11/3/2026 12:17:47 PM         4624 Information      An account was successfully logged on....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information      Credential Manager credentials were read....
```


Administrator: Windows PowerShell
11/3/2026 12:17:45 PM 5379 Information Credential Manager credentials were read....


Administrator: Windows PowerShell
11/3/2026 12:17:45 PM 5379 Information Credential Manager credentials were read....
11/3/2026 12:17:43 PM 5379 Information Credential Manager credentials were read....
11/3/2026 12:17:42 PM 4672 Information Special privileges assigned to new logon....
11/3/2026 12:17:42 PM 4624 Information An account was successfully logged on....
11/3/2026 12:13:07 PM 5379 Information Credential Manager credentials were read....

Command 2 (last hour):

$StartTime = (Get-Date).AddHours(-1)

Get-WinEvent -FilterHashtable @{

LogName='Security'

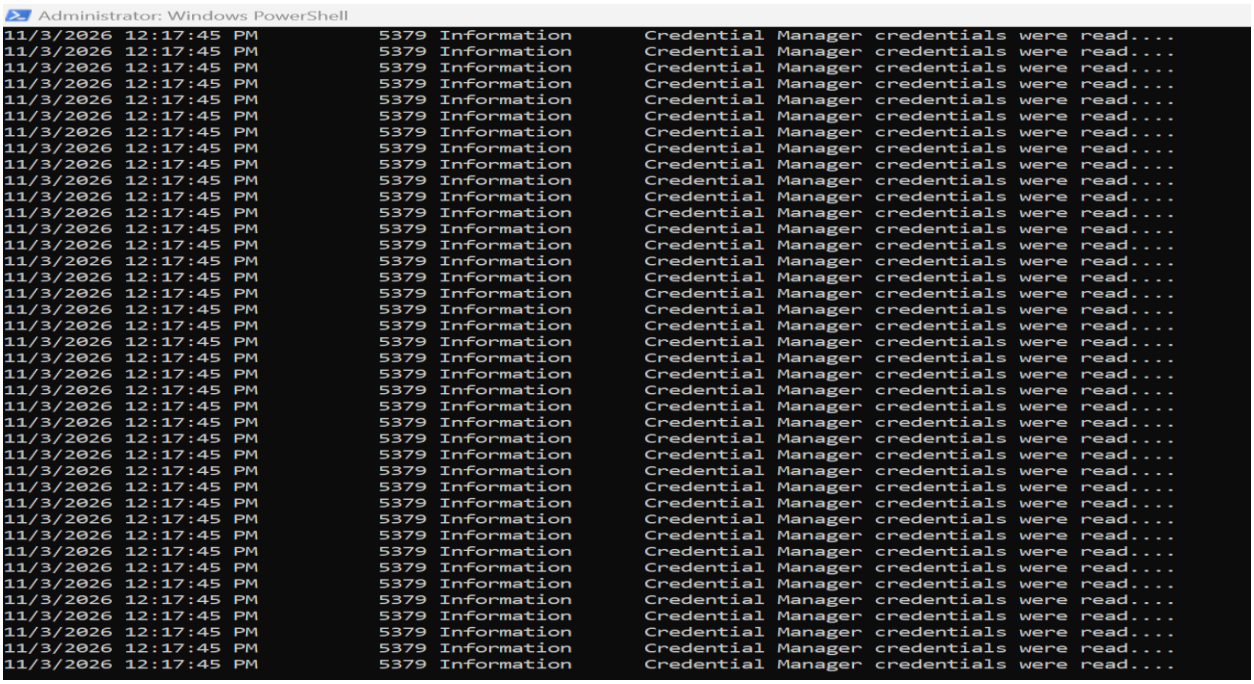
StartTime=$StartTime

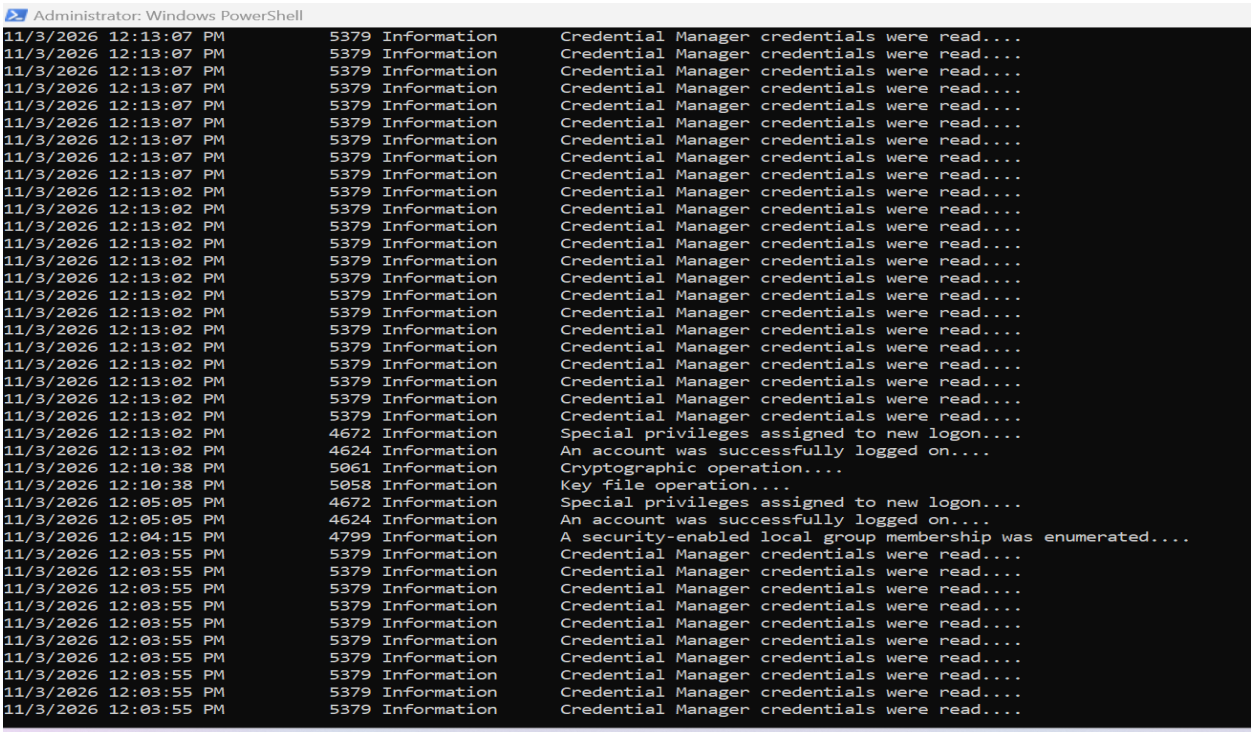
}

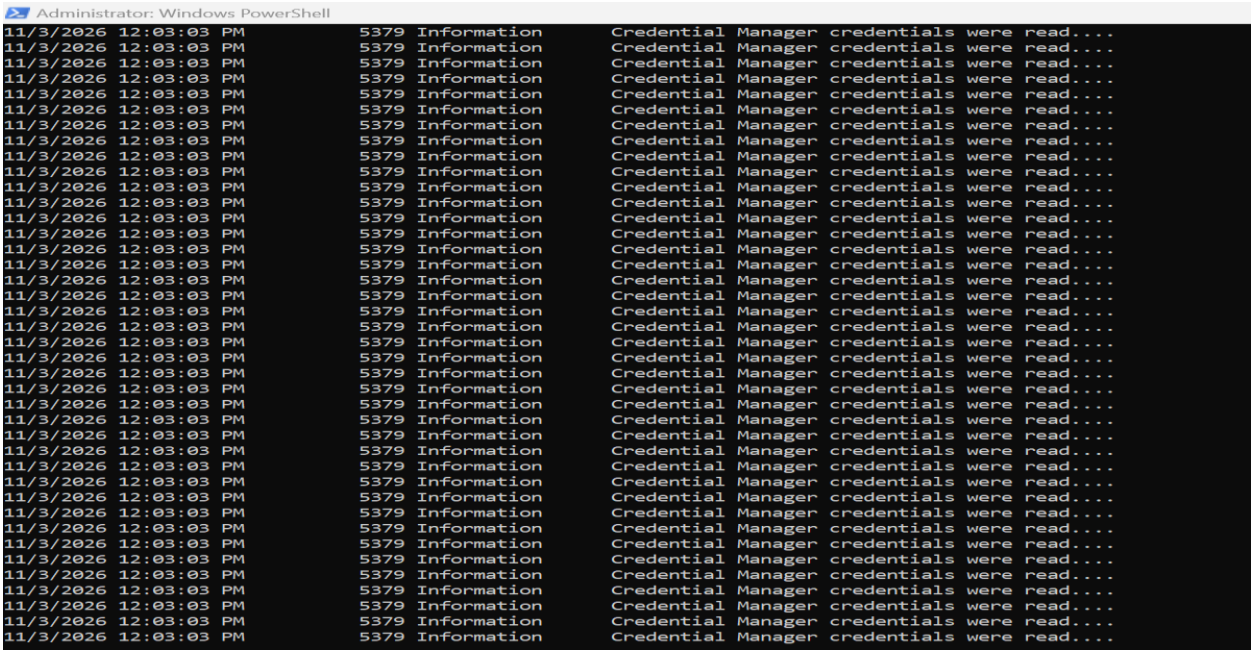
```
PS C:\Windows\system32> $StartTime = (Get-Date).AddHours(-1)
>> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     StartTime=$StartTime
>> }

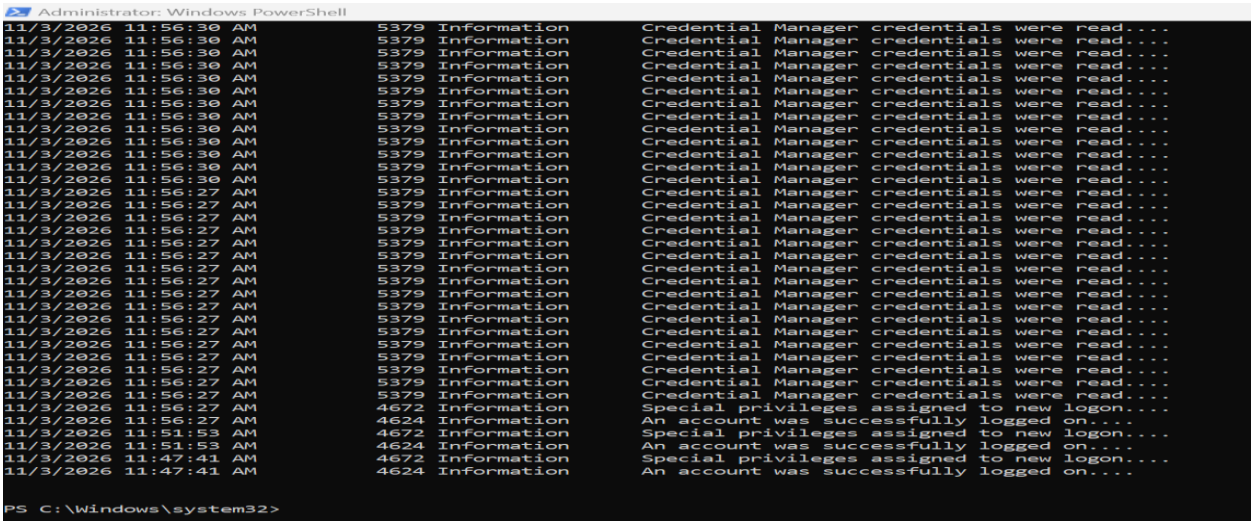

   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- --------------- -------
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:43:21 PM         4672 Information     Special privileges assigned to new logon....
11/3/2026 12:43:21 PM         4624 Information     An account was successfully logged on....
11/3/2026 12:37:04 PM         4672 Information     Special privileges assigned to new logon....
11/3/2026 12:37:04 PM         4624 Information     An account was successfully logged on....
11/3/2026 12:33:07 PM         4798 Information     A user's local group membership was enumerated....
11/3/2026 12:19:46 PM         4798 Information     A user's local group membership was enumerated....
11/3/2026 12:17:47 PM         4672 Information     Special privileges assigned to new logon....
11/3/2026 12:17:47 PM         4624 Information     An account was successfully logged on....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 12:17:45 PM         5379 Information     Credential Manager credentials were read....
```


Administrator: Windows PowerShell
11/3/2026 12:17:45 PM 5379 Information Credential Manager credentials were read....


Administrator: Windows PowerShell
11/3/2026 12:13:07 PM 5379 Information Credential Manager credentials were read....
11/3/2026 12:13:02 PM 5379 Information Credential Manager credentials were read....
11/3/2026 12:13:02 PM 4672 Information Special privileges assigned to new logon....
11/3/2026 12:13:02 PM 4624 Information An account was successfully logged on....
11/3/2026 12:10:38 PM 5061 Information Cryptographic operation....
11/3/2026 12:10:38 PM 5058 Information Key file operation....
11/3/2026 12:05:05 PM 4672 Information Special privileges assigned to new logon....
11/3/2026 12:05:05 PM 4624 Information An account was successfully logged on....
11/3/2026 12:04:15 PM 4799 Information A security-enabled local group membership was enumerated....
11/3/2026 12:03:55 PM 5379 Information Credential Manager credentials were read....





Explanation:

- (Get-Date).AddDays(-1) calculates the date and time 24 hours ago.

- (Get-Date).AddHours(-1) calculates the time one hour ago.

- StartTime=$StartTime filters events from that time forward.

- EndTime=$EndTime sets the upper limit, so only events inside the specified range are returned.

- -MaxEvents 100 limits the output to 100 events.

Observation:

The command returned events from the last 24 hours, including Event IDs ____, ____ and ____. The most recent event was recorded at ____. The last-hour query returned ____ events, and the specific date range query returned ____ events.

Exercise 4

Exercise 4: Count Events (Failed Logons in the Last 24 Hours):

$StartTime = (Get-Date).AddDays(-1)

$FailedLogons = Get-WinEvent -FilterHashtable @{

LogName='Security'

ID=4625

StartTime=$StartTime

} -ErrorAction SilentlyContinue

$FailedLogons.Count

```
PS C:\Windows\system32> $StartTime = (Get-Date).AddDays(-1)
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> }
>> $FailedLogons.Count
Get-WinEvent : No events were found that match the specified selection criteria.
At line:2 char:17
+ $FailedLogons = Get-WinEvent -FilterHashtable @{
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand

0
PS C:\Windows\system32>
```

Observation:

The command returned 0, and PowerShell reported "No events were found that match the specified selection criteria." This means there were no failed logon attempts (Event ID 4625) in the last 24 hours, because no wrong password had been entered yet.

Exercise 7: Analyze Failed Logon Attempts (Top 5 Source IP Addresses):

Command 1:

Get-WinEvent -FilterHashtable @{

LogName='Security'

ID=4625

StartTime=(Get-Date).AddDays(-1)

} | ForEach-Object {

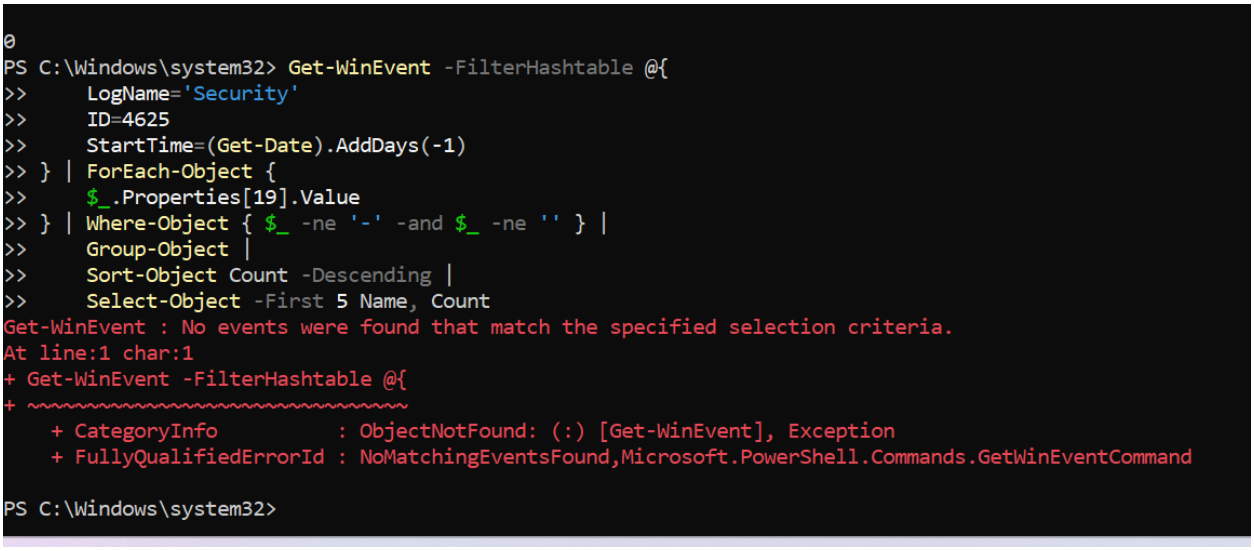
$_.Properties[19].Value

} | Where-Object { $_ -ne '-' -and $_ -ne '' } |

Group-Object |

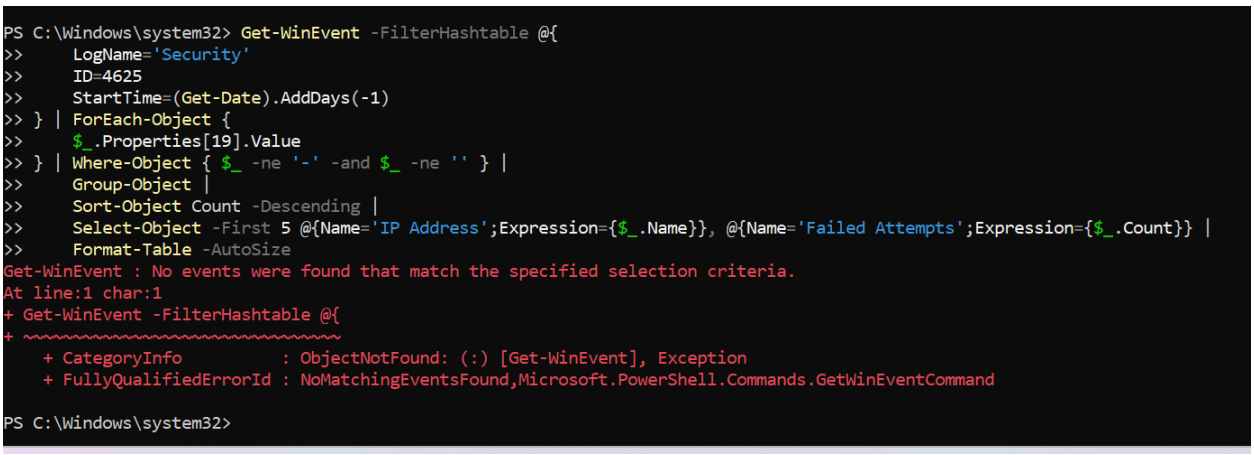
Sort-Object Count -Descending |

Select-Object -First 5 Name, Count



Command 2 (better formatted):

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $_.Properties[19].Value
} | Where-Object { $_ -ne '-' -and $_ -ne '' } |
    Group-Object |
    Sort-Object Count -Descending |
    Select-Object -First 5 @{Name='IP Address';Expression={$_.Name}}, @{Name='Failed Attempts';Expression={$_.Count}} |
    Format-Table -AutoSize
```



Explanation:

1. Get-WinEvent retrieves all failed logon events (Event ID 4625) from the last 24 hours.

2. ForEach-Object extracts the source IP address, which is stored at index 19 of the Properties array.

3. Where-Object removes empty values and "-", which appear when no IP address is recorded.

4. Group-Object groups the events by IP address.

5. Sort-Object Count -Descending sorts the groups from the highest count to the lowest.

6. Select-Object -First 5 selects the top 5 IP addresses.

7. In the second command, calculated properties rename the columns to "IP Address" and "Failed Attempts" for a cleaner table.

Observation:

The top source IP address was ____ with ____ failed attempts, followed by ____ with ____ attempts. This shows that most failed logons came from ____. A high number of failures from a single IP address can indicate a brute-force attack.

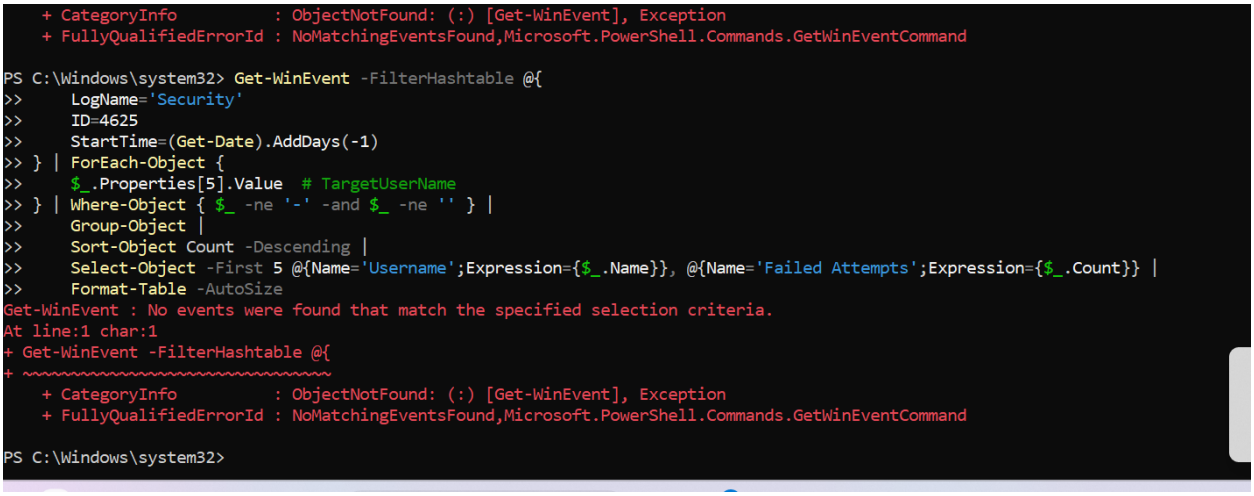
Observation:

No external source IP addresses were recorded because the failed logon attempts were generated locally on the machine, where the IP address field is empty ("-").

Exercise 8: Identify Targeted Accounts (Top 5 Usernames):

Command:

```
Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=4625
   StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
   $_.Properties[5].Value  # TargetUserName
} | Where-Object { $_ -ne '-' -and $_ -ne '' } |
   Group-Object |
   Sort-Object Count -Descending |
   Select-Object -First 5 @{Name='Username';Expression={$_.Name}}, @{Name='Failed Attempts';Expression={$_.Count}} |
   Format-Table -AutoSize
```



Explanation:

1. The command retrieves failed logon events (Event ID 4625) from the last 24 hours.

2. $_.Properties[5].Value extracts the TargetUserName, which is the account that failed to log on.

3. Where-Object filters out empty values and "-".

4. Group-Object groups the events by username.

5. Sort-Object Count -Descending orders the usernames by the number of failed attempts.

6. Select-Object -First 5 shows the top 5 targeted usernames in a table with the columns "Username" and "Failed Attempts".

Observation:

The most targeted username was ____ with ____ failed attempts, followed by ____ with ____ attempts. Repeated failures against the same account may indicate a password guessing or brute-force attempt against that user.

Observation:

No events were found because no failed logon attempts (Event ID 4625) had been generated in the last 24 hours at that time.

Exercise 11: Generate Test Failed Logons

Command 1

1..10 | ForEach-Object {

net use \\localhost\C$ /user:FakeUser WrongPassword 2>$null

Start-Sleep -Seconds 2

}

Command 2

Get-WinEvent -FilterHashtable @{

LogName='Security'

ID=4625

StartTime=(Get-Date).AddMinutes(-5)

} | Select-Object TimeCreated, Message | Format-List

```
PS C:\Windows\system32> 1..10 | ForEach-Object {
>>     net use \\localhost\C$ /user:FakeUser WrongPassword 2>$null
>>     Start-Sleep -Seconds 2
>> }
PS C:\Windows\system32>
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddMinutes(-5)
>> } | Select-Object TimeCreated, Message | Format-List


TimeCreated : 11/3/2026 2:19:48 PM
Message     : An account failed to log on.

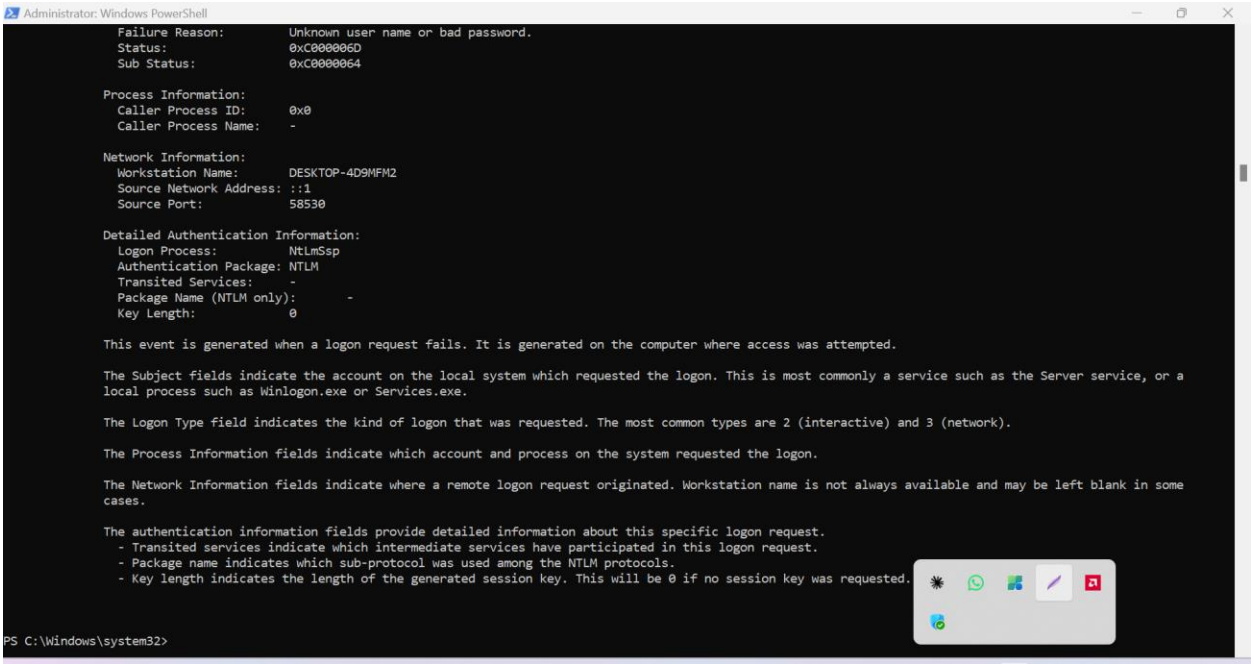
              Subject:
                Security ID:            S-1-0-0
                Account Name:           -
                Account Domain:         -
                Logon ID:               0x0

              Logon Type:                       3

              Account For Which Logon Failed:
                Security ID:            S-1-0-0
                Account Name:           FakeUser
                Account Domain:         -

              Failure Information:
                Failure Reason:         Unknown user name or bad password.
                Status:                 0xC000006D
                Sub Status:             0xC0000064

              Process Information:
                Caller Process ID:      0x0
                Caller Process Name:    -
```



Explanation:

- 1..10 creates a loop that runs 10 times.

- net use \\localhost\C$ /user:FakeUser WrongPassword tries to access a network share with a fake username and wrong password, which generates Event ID 4625 (failed logon).

- 2>$null hides the error messages.

- Start-Sleep -Seconds 2 waits 2 seconds between attempts.

- The second command lists the failed logon events created in the last 5 minutes to verify that the events were generated.

Observation:

The loop generated ____ failed logon events (Event ID 4625) within the last 5 minutes. The message of each event showed "An account failed to log on" with the account name FakeUser.

Exercise 9: Analyze Logon Types

Command:

```
Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=4625
   StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
   $LogonType = $_.Properties[10].Value
   switch ($LogonType) {
      2 { 'Interactive' }
      3 { 'Network' }
      4 { 'Batch' }
      5 { 'Service' }
      7 { 'Unlock' }
      8 { 'NetworkCleartext' }
      9 { 'NewCredentials' }
      10 { 'RemoteInteractive' }
      11 { 'CachedInteractive' }
      default { "Unknown ($LogonType)" }
   }
} | Group-Object |
   Sort-Object Count -Descending |
   Select-Object @{Name='Logon Type';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} |
   Format-Table -AutoSize
```

Output:

```
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $LogonType = $_.Properties[10].Value
>>     switch ($LogonType) {
>>         2 { 'Interactive' }
>>         3 { 'Network' }
>>         4 { 'Batch' }
>>         5 { 'Service' }
>>         7 { 'Unlock' }
>>         8 { 'NetworkCleartext' }
>>         9 { 'NewCredentials' }
>>         10 { 'RemoteInteractive' }
>>         11 { 'CachedInteractive' }
>>         default { "Unknown ($LogonType)" }
>>     }
>> } | Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object @{Name='Logon Type';Expression={$_.Name}}, @{Name='Count';Expression={$_.Count}} |
>>     Format-Table -AutoSize

Logon Type Count
---------- -----
Network       19


PS C:\Windows\system32>
```

Explanation:

- The command retrieves failed logon events (Event ID 4625) from the last 24 hours.

- Properties[10] holds the numeric logon type of each event.

- The switch statement converts each number into its name (for example, 3 = Network, 10 = RemoteInteractive).

- Group-Object counts how many events belong to each logon type.

- Sort-Object Count -Descending shows the most common logon type first.

- Format-Table -AutoSize displays the result as a table.

Observation:

The most common logon type for failed attempts was ____ (____) with ____ events. Logon type 3 (Network) usually indicates attempts through network connections such as file shares, while type 10 (RemoteInteractive) indicates RDP attempts.

Exercise 10: Identify Brute-Force Patterns

Script (Detect-BruteForce.ps1):

```
# Define threshold
$FailureThreshold = 5
$TimeWindow = (Get-Date).AddHours(-1)

# Get failed logons
$FailedLogons = Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=4625
   StartTime=$TimeWindow
} | ForEach-Object {
   [PSCustomObject]@{
      TimeCreated = $_.TimeCreated
      SourceIP = $_.Properties[19].Value
      TargetUser = $_.Properties[5].Value
      LogonType = $_.Properties[10].Value
   }
}

# Group by source IP
$SuspiciousIPs = $FailedLogons |
   Where-Object { $_.SourceIP -ne '-' } |
   Group-Object SourceIP |
   Where-Object { $_.Count -ge $FailureThreshold } |
   Sort-Object Count -Descending
```

```
# Display results
Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow

foreach ($IP in $SuspiciousIPs) {
    Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
    Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
    Write-Host "Targeted Accounts:" -ForegroundColor Yellow
    $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
        Write-Host "  - $_"
    }
    Write-Host ""
}
```

Output:

```
Logon Type Count
---------- -----
Network        19


PS C:\Windows\system32> # Define threshold
>> $FailureThreshold = 5
>> $TimeWindow = (Get-Date).AddHours(-1)
>>
>> # Get failed logons
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$TimeWindow
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[10].Value
>>     }
>> }
>>
>> # Group by source IP
>> $SuspiciousIPs = $FailedLogons |
>>     Where-Object { $_.SourceIP -ne '-' } |
>>     Group-Object SourceIP |
>>     Where-Object { $_.Count -ge $FailureThreshold } |
>>     Sort-Object Count -Descending
>>
>> # Display results
>> Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
>> Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
>> Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow
>>
>> foreach ($IP in $SuspiciousIPs) {
>>     Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
```

Select Administrator: Windows PowerShell

```
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[10].Value
>>     }
>> }
>>
>> # Group by source IP
>> $SuspiciousIPs = $FailedLogons |
>>     Where-Object { $_.SourceIP -ne '-' } |
>>     Group-Object SourceIP |
>>     Where-Object { $_.Count -ge $FailureThreshold } |
>>     Sort-Object Count -Descending
>>
>> # Display results
>> Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
>> Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
>> Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow
>>
>> foreach ($IP in $SuspiciousIPs) {
>>     Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
>>     Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
>>     Write-Host "Targeted Accounts:" -ForegroundColor Yellow
>>     $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
>>         Write-Host "  - $_"
>>     }
>>     Write-Host ""
>> }

=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===
Time Window: Last 1 hour
Threshold: 5 failed attempts

Source IP: ::1
Failed Attempts: 19
Targeted Accounts:
  - FakeUser

PS C:\Windows\system32>
```

Explanation:

- $FailureThreshold = 5 sets the minimum number of failed attempts needed to flag an IP address.

- $TimeWindow = (Get-Date).AddHours(-1) limits the analysis to the last hour.

- The script collects failed logon events (Event ID 4625) and extracts the time, source IP, target username and logon type.

- Group-Object SourceIP groups the events by source IP address.

- Where-Object { $_.Count -ge $FailureThreshold } keeps only IP addresses with 5 or more failures.

- The foreach loop prints each suspicious IP, its number of failed attempts, and the targeted accounts.

- Write-Host with -ForegroundColor displays the output in colors (red for alerts, yellow for details).

Observation:

The script detected ____ suspicious IP address(es). Source IP ____ had ____ failed attempts and targeted the account(s) ____.

Observation:

No suspicious IP addresses were listed because the failed logon attempts were generated locally, where the source IP field is empty ("-"). The script correctly ignores such entries.

Exercise 12: Detect Explicit Credential Usage (Event ID 4648):

Command:

Get-WinEvent -FilterHashtable @{

```
    LogName='Security'
    ID=4648
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        SourceUser = $xml.Event.EventData.Data[1].'#text'
        TargetUser = $xml.Event.EventData.Data[5].'#text'
        TargetServer = $xml.Event.EventData.Data[8].'#text'
        Process = $xml.Event.EventData.Data[11].'#text'
    }
} | Format-Table -AutoSize
```

Output:

Administrator: Windows PowerShell

```
PS C:\Windows\system32> # Define threshold
>> $FailureThreshold = 5
>> $TimeWindow = (Get-Date).AddHours(-1)
>>
>> # Get failed logons
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$TimeWindow
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[10].Value
>>     }
>> }
>>
>> # Group by source IP
>> $SuspiciousIPs = $FailedLogons |
>>     Where-Object { $_.SourceIP -ne '-' } |
>>     Group-Object SourceIP |
>>     Where-Object { $_.Count -ge $FailureThreshold } |
>>     Sort-Object Count -Descending
>>
>> # Display results
>> Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
>> Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
>> Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow
>>
>> foreach ($IP in $SuspiciousIPs) {

>>     Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
>>     Write-Host "Targeted Accounts:" -ForegroundColor Yellow
```

Select Administrator: Windows PowerShell

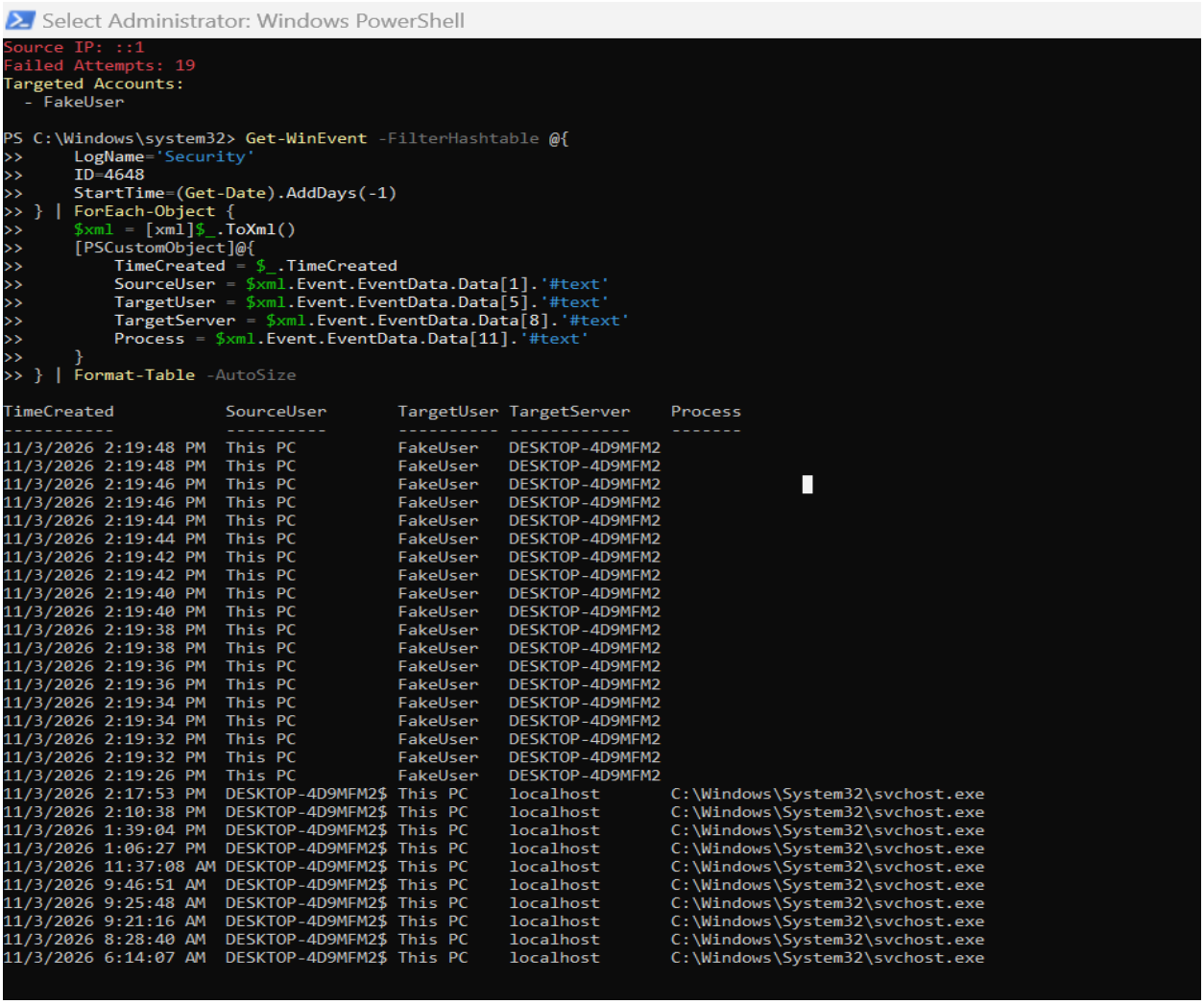
```
>> # Display results
>> Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
>> Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
>> Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow
>>
>> foreach ($IP in $SuspiciousIPs) {
>>     Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
>>     Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
>>     Write-Host "Targeted Accounts:" -ForegroundColor Yellow
>>     $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
>>         Write-Host "  - $_"
>>     }
>>     Write-Host ""
>> }

=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===
Time Window: Last 1 hour
Threshold: 5 failed attempts

Source IP: ::1
Failed Attempts: 19
Targeted Accounts:
  - FakeUser

PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4648
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceUser = $xml.Event.EventData.Data[1].'#text'
>>         TargetUser = $xml.Event.EventData.Data[5].'#text'
>>         TargetServer = $xml.Event.EventData.Data[8].'#text'
>>         Process = $xml.Event.EventData.Data[11].'#text'
>>     }
>> } | Format-Table -AutoSize

TimeCreated           SourceUser       TargetUser TargetServer    Process
```



Explanation:

- Event ID 4648 is logged when a logon is attempted using explicit credentials (for example RunAs or PsExec), which is common in lateral movement.

- ToXml() converts each event into XML so the data fields can be read.

- Data[1] is the account that used the credentials, Data[5] is the target account, Data[8] is the target server, and Data[11] is the process that made the request.

- A custom object is created for each event and displayed as a table with Format-Table -AutoSize.

Observation:

The query returned ____ events. The most common source user was ____, the target server was ____ and the process was ____. Explicit credential use by unexpected users or processes toward other servers can indicate lateral movement.

Exercise 13: Detect Network Share Access (Event ID 5140)

Command:

```
Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=5140
   StartTime=(Get-Date).AddHours(-1)
} | ForEach-Object {
   $xml = [xml]$_.ToXml()
   [PSCustomObject]@{
      TimeCreated = $_.TimeCreated
      Account = $xml.Event.EventData.Data[1].'#text'
      SourceIP = $xml.Event.EventData.Data[5].'#text'
      ShareName = $xml.Event.EventData.Data[7].'#text'
   }
} | Format-Table -AutoSize
```

Output:

```
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=5140
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         SourceIP = $xml.Event.EventData.Data[5].'#text'
>>         ShareName = $xml.Event.EventData.Data[7].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand

PS C:\Windows\system32>
```

Explanation:

- Event ID 5140 is logged when a network share is accessed, which is useful for detecting lateral movement.

- The command reads the events from the last hour and converts each one to XML.

- Data[1] is the account name, Data[5] is the source IP address, and Data[7] is the share name (for example *\C$).

- The results are shown as a table with Format-Table -AutoSize.

Observation:

The query returned ____ network share access events. The account ____ accessed the share ____ from the source IP ____. Access to administrative shares such as C$ or ADMIN$ from unexpected accounts or IP addresses may indicate lateral movement.

Observation:

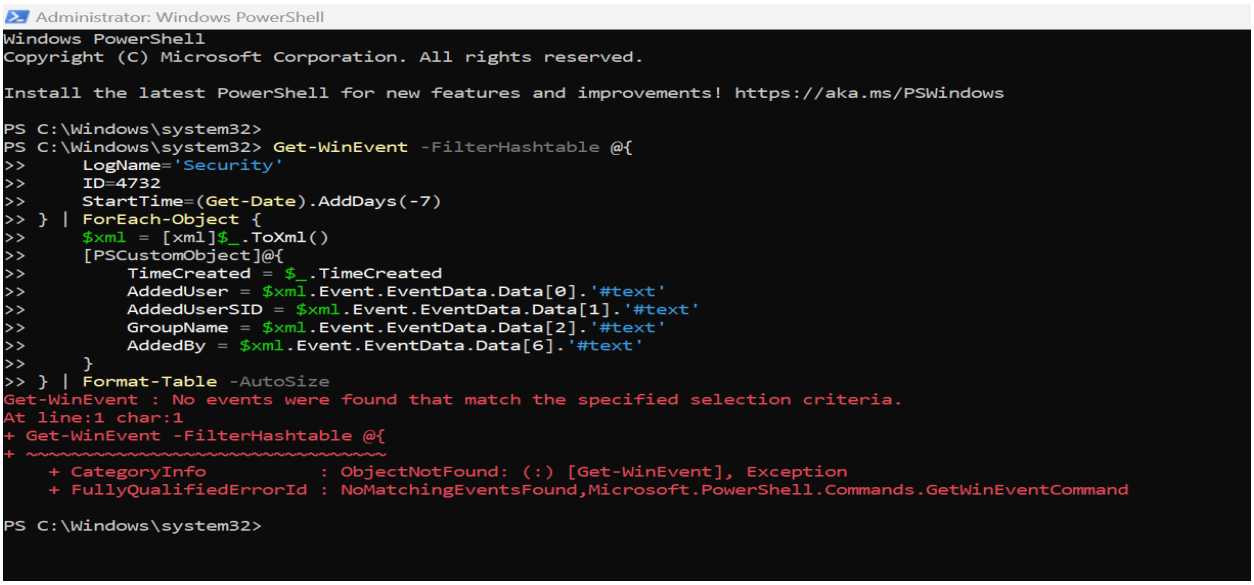
No network share access events (Event ID 5140) were found in the last hour, so no lateral movement through network shares was detected.

Exercise 14: Monitor Admin Group Changes (Event ID 4732):

Command:

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4732
    StartTime=(Get-Date).AddDays(-7)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        AddedUser = $xml.Event.EventData.Data[0].'#text'
        AddedUserSID = $xml.Event.EventData.Data[1].'#text'
        GroupName = $xml.Event.EventData.Data[2].'#text'
        AddedBy = $xml.Event.EventData.Data[6].'#text'
    }
} | Format-Table -AutoSize
```

Output:



Explanation:

- Event ID 4732 is logged when a member is added to a security-enabled local group, such as Administrators.

- The query looks at the last 7 days of the Security log.

- ToXml() converts each event to XML so the data fields can be read.

- Data[0] is the added member name, Data[1] is the member SID, Data[2] is the group name, and Data[6] is the account that made the change.

- The results are displayed as a table with Format-Table -AutoSize.

Observation:

The query returned ____ events. The account ____ was added to the group ____ by ____ at ____. Unexpected additions to the Administrators group may indicate privilege escalation.

Observation:

No events were found because no member had been added to a local group in the last 7 days, so no privilege escalation through group changes was detected.

Exercise 15: Detect Special Privilege Assignments (Event ID 4672)

Command:

```
Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=4672
   StartTime=(Get-Date).AddHours(-1)
} | ForEach-Object {
   $xml = [xml]$_.ToXml()
   [PSCustomObject]@{
      TimeCreated = $_.TimeCreated
      Account = $xml.Event.EventData.Data[1].'#text'
```

LogonID = $xml.Event.EventData.Data[3].'#text'

}

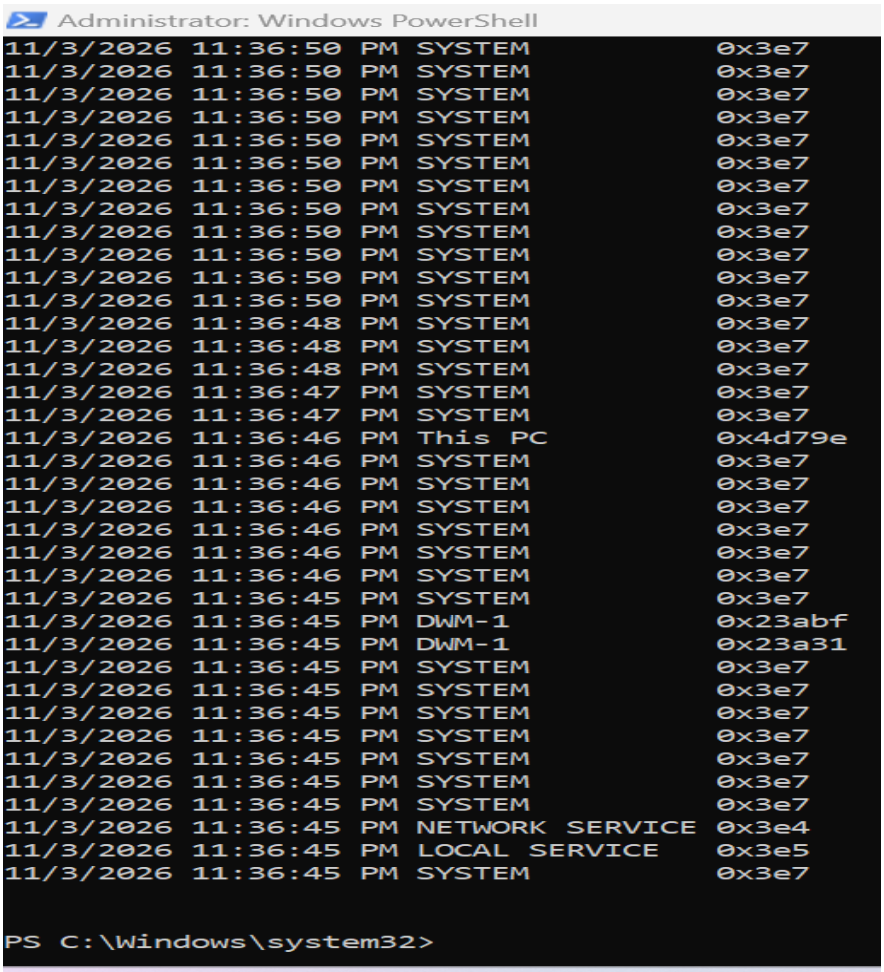
} | Where-Object { $_.Account -notlike '*$' } | # Filter out system accounts

Format-Table -AutoSize

Output:

```
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{
>>      LogName='Security'
>>      ID=4672
>>      StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>      $xml = [xml]$_.ToXml()
>>      [PSCustomObject]@{
>>          TimeCreated = $_.TimeCreated
>>          Account = $xml.Event.EventData.Data[1].'#text'
>>          LogonID = $xml.Event.EventData.Data[3].'#text'
>>      }
>> } | Where-Object { $_.Account -notlike '*$' } |  # Filter out system accounts
>>      Format-Table -AutoSize

TimeCreated           Account         LogonID
-----------           -------         -------
11/4/2026 12:00:54 AM SYSTEM          0x3e7
11/3/2026 11:56:38 PM SYSTEM          0x3e7
11/3/2026 11:51:48 PM SYSTEM          0x3e7
11/3/2026 11:51:47 PM SYSTEM          0x3e7
11/3/2026 11:51:47 PM SYSTEM          0x3e7
11/3/2026 11:51:47 PM SYSTEM          0x3e7
11/3/2026 11:49:47 PM SYSTEM          0x3e7
11/3/2026 11:47:08 PM SYSTEM          0x3e7
11/3/2026 11:47:05 PM SYSTEM          0x3e7
11/3/2026 11:45:30 PM SYSTEM          0x3e7
11/3/2026 11:45:27 PM SYSTEM          0x3e7
11/3/2026 11:41:48 PM SYSTEM          0x3e7
11/3/2026 11:40:38 PM SYSTEM          0x3e7
11/3/2026 11:38:52 PM SYSTEM          0x3e7
11/3/2026 11:38:52 PM SYSTEM          0x3e7
11/3/2026 11:38:52 PM SYSTEM          0x3e7
11/3/2026 11:38:39 PM SYSTEM          0x3e7
11/3/2026 11:37:31 PM SYSTEM          0x3e7
11/3/2026 11:37:04 PM SYSTEM          0x3e7
11/3/2026 11:37:04 PM SYSTEM          0x3e7
11/3/2026 11:36:58 PM SYSTEM          0x3e7
11/3/2026 11:36:57 PM SYSTEM          0x3e7
```



Explanation:

- Event ID 4672 is logged when special privileges (such as administrator-level rights) are assigned to a new logon session.

- The query checks the last hour of the Security log.

- Data[1] is the account name and Data[3] is the logon ID of the session.

- Where-Object { $_.Account -notlike '*$' } removes computer (machine) accounts, whose names end with a dollar sign.

- The results are displayed as a table with Format-Table -AutoSize.

Observation:

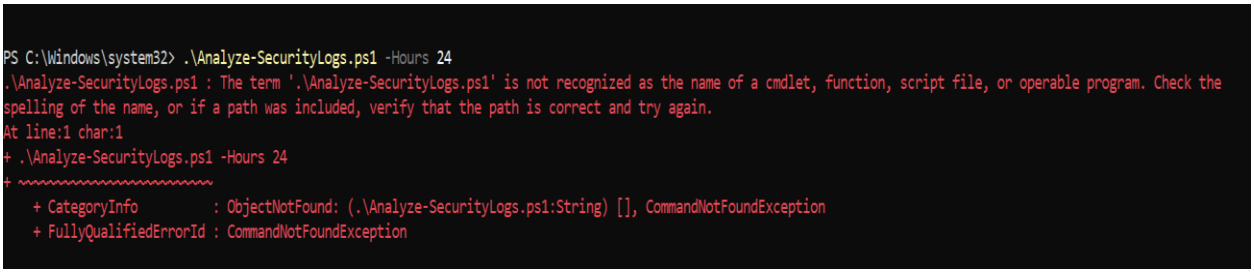
The query returned ____ events in the last hour. Special privileges were assigned to the account(s) ____. System accounts such as SYSTEM and LOCAL SERVICE commonly appear here, while an unexpected user account receiving special privileges should be investigated.

Exercise 16: Create a Complete Security Analysis Script

Create a file:

Command:

.\Analyze-SecurityLogs.ps1 -Hours 24



Command:

```
<#
.SYNOPSIS
    Comprehensive Windows Security Log Analysis Script
.DESCRIPTION
    Analyzes Windows Security Event Logs for suspicious activity
.PARAMETER Hours
    Number of hours to analyze (default: 24)
.EXAMPLE
    .\Analyze-SecurityLogs.ps1 -Hours 24
#>

param(
    [int]$Hours = 24
)

$StartTime = (Get-Date).AddHours(-$Hours)
```

```
Write-Host "`n========================================" -ForegroundColor Cyan
Write-Host "Windows Security Log Analysis Report" -ForegroundColor Cyan
Write-Host "========================================" -ForegroundColor Cyan
Write-Host "Analysis Period: Last $Hours hours" -ForegroundColor Yellow
Write-Host "Start Time: $StartTime" -ForegroundColor Yellow
Write-Host "========================================`n" -ForegroundColor Cyan

# 1. Failed Logon Analysis
Write-Host "[1] FAILED LOGON ANALYSIS" -ForegroundColor Green
Write-Host "----------------------------------" -ForegroundColor Green

$FailedLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($FailedLogons) {
    Write-Host "Total Failed Logons: $($FailedLogons.Count)" -ForegroundColor Yellow

    Write-Host "`nTop 5 Source IPs:" -ForegroundColor Cyan
    $FailedLogons | ForEach-Object { $_.Properties[19].Value } |
        Where-Object { $_ -ne '-' } |
        Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
        Format-Table @{Name='IP Address';Expression={$_.Name}},
@{Name='Attempts';Expression={$_.Count}} -AutoSize
```

```
    Write-Host "Top 5 Targeted Accounts:" -ForegroundColor Cyan
    $FailedLogons | ForEach-Object { $_.Properties[5].Value } |
        Where-Object { $_ -ne '-' } |
        Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
        Format-Table @{Name='Username';Expression={$_.Name}},
@{Name='Attempts';Expression={$_.Count}} -AutoSize
} else {
    Write-Host "No failed logons found." -ForegroundColor Green
}

# 2. Successful Logon Analysis
Write-Host "`n[2] SUCCESSFUL LOGON ANALYSIS" -ForegroundColor Green
Write-Host "-----------------------------------" -ForegroundColor Green

$SuccessfulLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4624
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($SuccessfulLogons) {
    Write-Host "Total Successful Logons: $($SuccessfulLogons.Count)" -ForegroundColor Yellow

    Write-Host "`nLogon Type Distribution:" -ForegroundColor Cyan
    $SuccessfulLogons | ForEach-Object {
        $LogonType = $_.Properties[8].Value
```

```
        switch ($LogonType) {
            2 { 'Interactive' }
            3 { 'Network' }
            10 { 'RemoteInteractive' }
            default { "Type $LogonType" }
        }
    } | Group-Object | Sort-Object Count -Descending |
        Format-Table @{Name='Logon Type';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
}

# 3. Account Management
Write-Host "`n[3] ACCOUNT MANAGEMENT EVENTS" -ForegroundColor Green
Write-Host "-----------------------------------" -ForegroundColor Green

$AccountEvents = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4720,4722,4724,4732
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($AccountEvents) {
    Write-Host "Account Management Events: $($AccountEvents.Count)" -ForegroundColor
Yellow
    $AccountEvents | Group-Object Id |
        Format-Table @{Name='Event ID';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
} else {
```

```
    Write-Host "No account management events found." -ForegroundColor Green
}

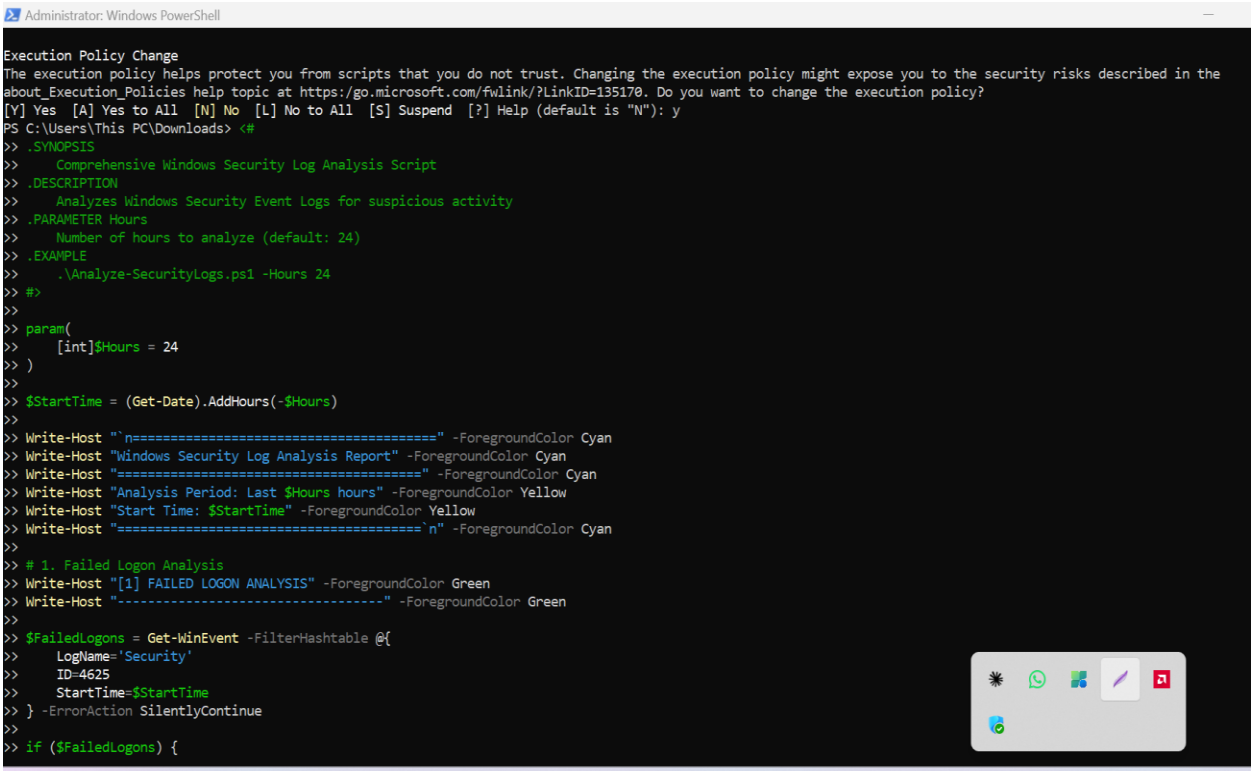
# 4. Privilege Escalation
Write-Host "`n[4] PRIVILEGE ESCALATION INDICATORS" -ForegroundColor Green
Write-Host "-----------------------------------" -ForegroundColor Green

$PrivEsc = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4672,4648
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($PrivEsc) {
    Write-Host "Privilege-Related Events: $($PrivEsc.Count)" -ForegroundColor Yellow
    $PrivEsc | Group-Object Id |
        Format-Table @{Name='Event ID';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
}

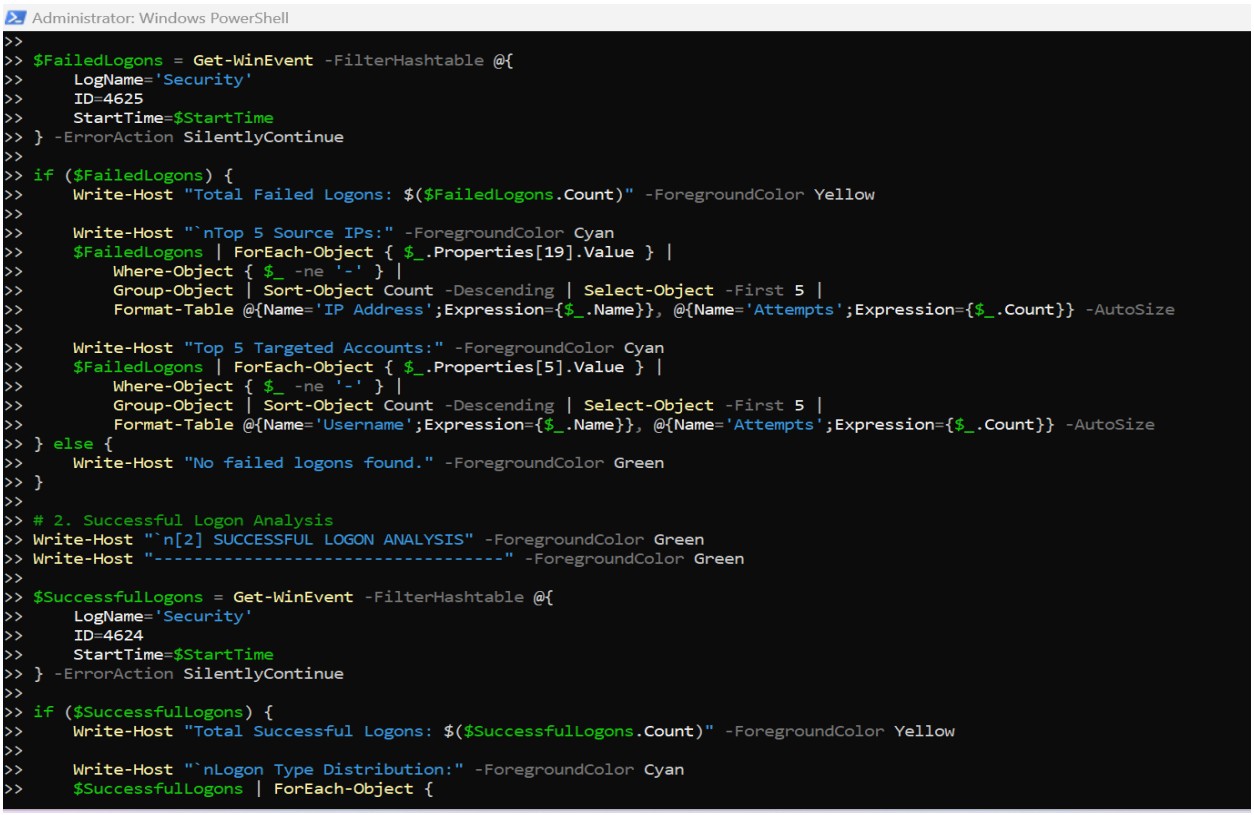
Write-Host "`n========================================" -ForegroundColor Cyan
Write-Host "Analysis Complete" -ForegroundColor Cyan
Write-Host "========================================`n" -ForegroundColor Cyan
```

Output:



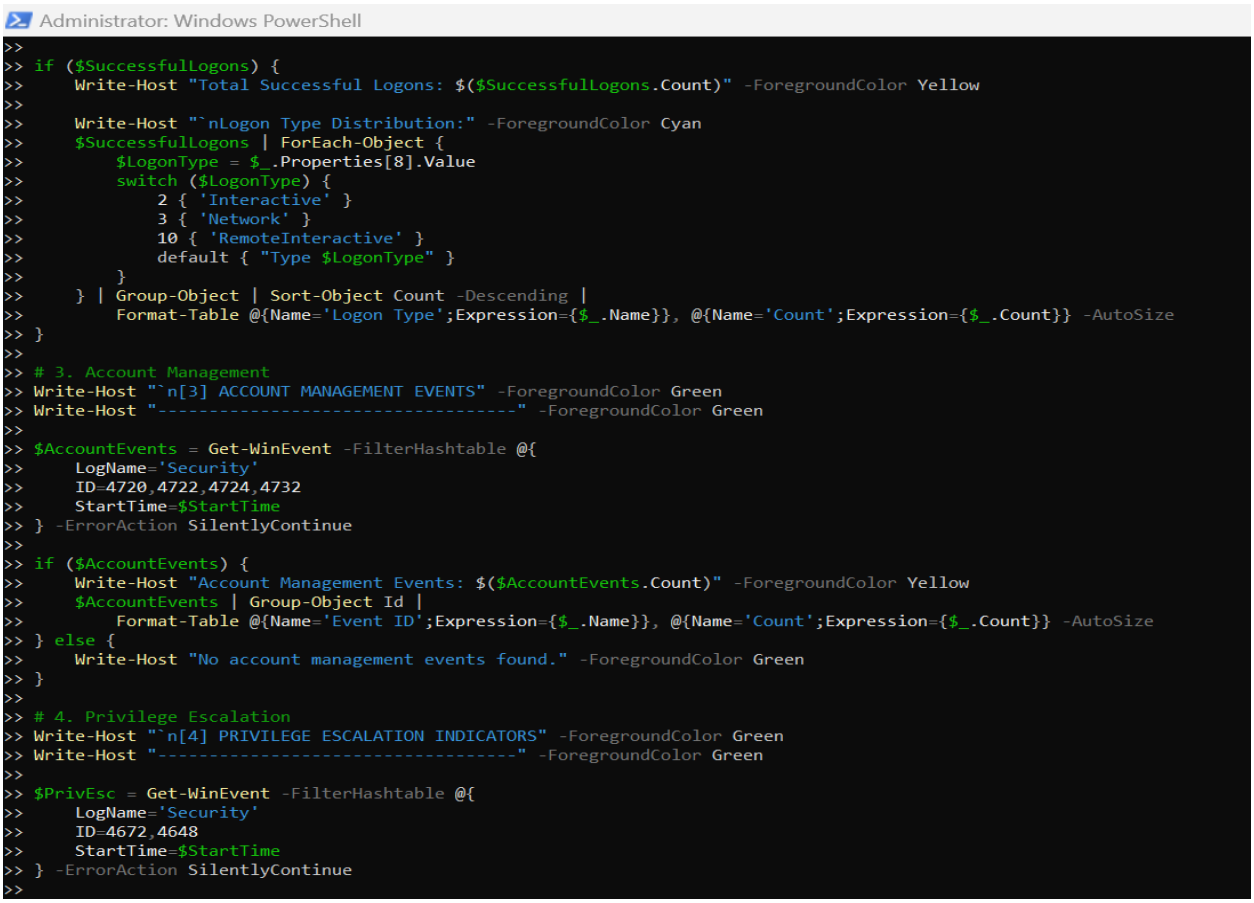
```
Administrator: Windows PowerShell

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution policy might expose you to the security risks described in the
about_Execution_Policies help topic at https:/go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y
PS C:\Users\This PC\Downloads> <#
>> .SYNOPSIS
>>     Comprehensive Windows Security Log Analysis Script
>> .DESCRIPTION
>>     Analyzes Windows Security Event Logs for suspicious activity
>> .PARAMETER Hours
>>     Number of hours to analyze (default: 24)
>> .EXAMPLE
>>     .\Analyze-SecurityLogs.ps1 -Hours 24
>> #>
>>
>> param(
>>     [int]$Hours = 24
>> )
>>
>> $StartTime = (Get-Date).AddHours(-$Hours)
>>
>> Write-Host "`n========================================" -ForegroundColor Cyan
>> Write-Host "Windows Security Log Analysis Report" -ForegroundColor Cyan
>> Write-Host "========================================" -ForegroundColor Cyan
>> Write-Host "Analysis Period: Last $Hours hours" -ForegroundColor Yellow
>> Write-Host "Start Time: $StartTime" -ForegroundColor Yellow
>> Write-Host "========================================`n" -ForegroundColor Cyan
>>
>> # 1. Failed Logon Analysis
>> Write-Host "[1] FAILED LOGON ANALYSIS" -ForegroundColor Green
>> Write-Host "----------------------------------" -ForegroundColor Green
>>
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($FailedLogons) {
```



```
Administrator: Windows PowerShell

>>
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($FailedLogons) {
>>     Write-Host "Total Failed Logons: $($FailedLogons.Count)" -ForegroundColor Yellow
>>
>>     Write-Host "`nTop 5 Source IPs:" -ForegroundColor Cyan
>>     $FailedLogons | ForEach-Object { $_.Properties[19].Value } |
>>         Where-Object { $_ -ne '-' } |
>>         Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
>>         Format-Table @{Name='IP Address';Expression={$_.Name}}, @{Name='Attempts';Expression={$_.Count}} -AutoSize
>>
>>     Write-Host "Top 5 Targeted Accounts:" -ForegroundColor Cyan
>>     $FailedLogons | ForEach-Object { $_.Properties[5].Value } |
>>         Where-Object { $_ -ne '-' } |
>>         Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
>>         Format-Table @{Name='Username';Expression={$_.Name}}, @{Name='Attempts';Expression={$_.Count}} -AutoSize
>> } else {
>>     Write-Host "No failed logons found." -ForegroundColor Green
>> }
>>
>> # 2. Successful Logon Analysis
>> Write-Host "`n[2] SUCCESSFUL LOGON ANALYSIS" -ForegroundColor Green
>> Write-Host "----------------------------------" -ForegroundColor Green
>>
>> $SuccessfulLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4624
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($SuccessfulLogons) {
>>     Write-Host "Total Successful Logons: $($SuccessfulLogons.Count)" -ForegroundColor Yellow
>>
>>     Write-Host "`nLogon Type Distribution:" -ForegroundColor Cyan
>>     $SuccessfulLogons | ForEach-Object {
```



Explanation:

- param([int]$Hours = 24) lets the user choose how many hours to analyze; the default is 24.

- $StartTime calculates the beginning of the analysis period.

- Section 1 counts failed logons (Event ID 4625) and lists the top 5 source IPs and the top 5 targeted accounts.

- Section 2 counts successful logons (Event ID 4624) and shows the distribution of logon types (Interactive, Network, RemoteInteractive).

- Section 3 counts account management events (Event IDs 4720, 4722, 4724 and 4732).

- Section 4 counts privilege-related events (Event IDs 4672 and 4648), which are indicators of privilege escalation.

- -ErrorAction SilentlyContinue prevents red errors when no matching events exist.

- Write-Host with colors makes the report easy to read.

Observation:

The script analyzed the last 24 hours and found ____ failed logons and ____ successful logons. The most common logon type was ____. There were ____ account management events and ____ privilege-related events. The report shows that ____.

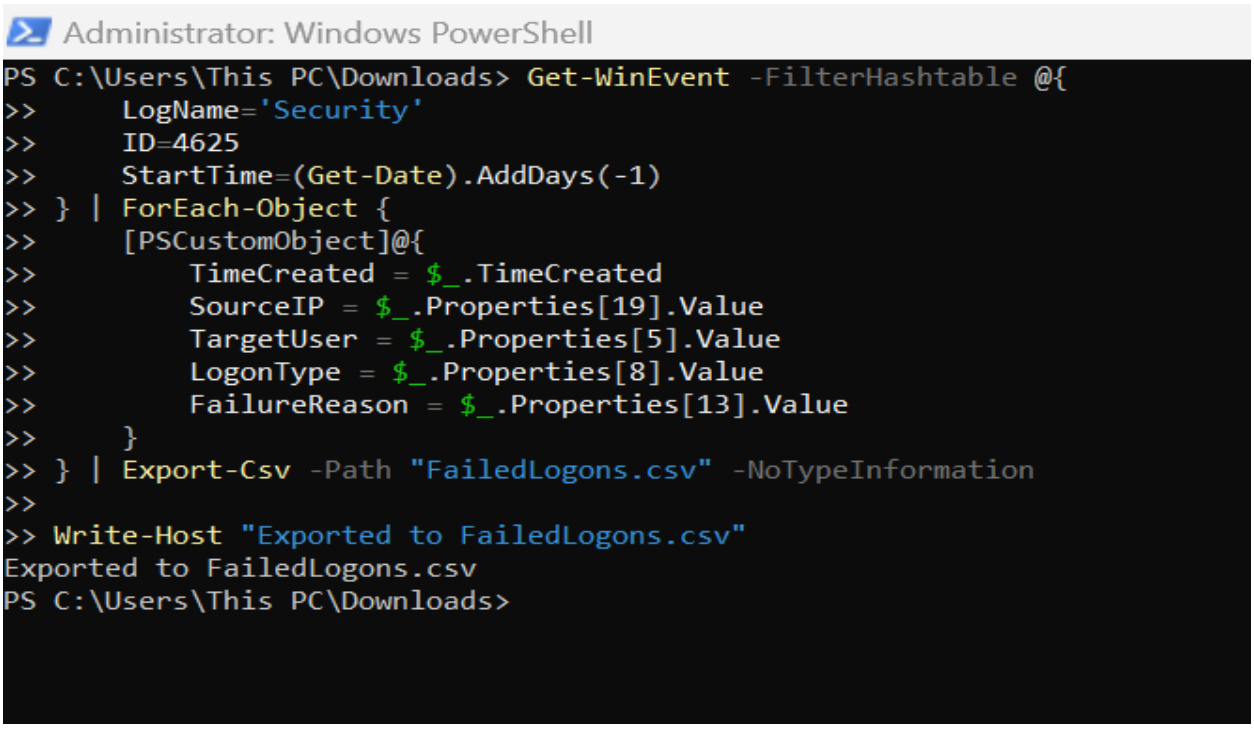
Exercise 17: Export to CSV

Command:

```
Get-WinEvent -FilterHashtable @{
   LogName='Security'
   ID=4625
   StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
   [PSCustomObject]@{
      TimeCreated = $_.TimeCreated
      SourceIP = $_.Properties[19].Value
      TargetUser = $_.Properties[5].Value
      LogonType = $_.Properties[8].Value
      FailureReason = $_.Properties[13].Value
   }
} | Export-Csv -Path "FailedLogons.csv" -NoTypeInformation

Write-Host "Exported to FailedLogons.csv"
```

Output:



Observation:

The script successfully parsed the recent failed logon attempts and exported the structured data into FailedLogons.csv. This CSV file can now be easily opened in Microsoft Excel or imported into other security analysis tools for deeper investigation.

Exercise 18: Export to HTML Report

Command:

$FailedLogons = Get-WinEvent -FilterHashtable @{

LogName='Security'

ID=4625

StartTime=(Get-Date).AddDays(-1)

} | ForEach-Object {

[PSCustomObject]@{

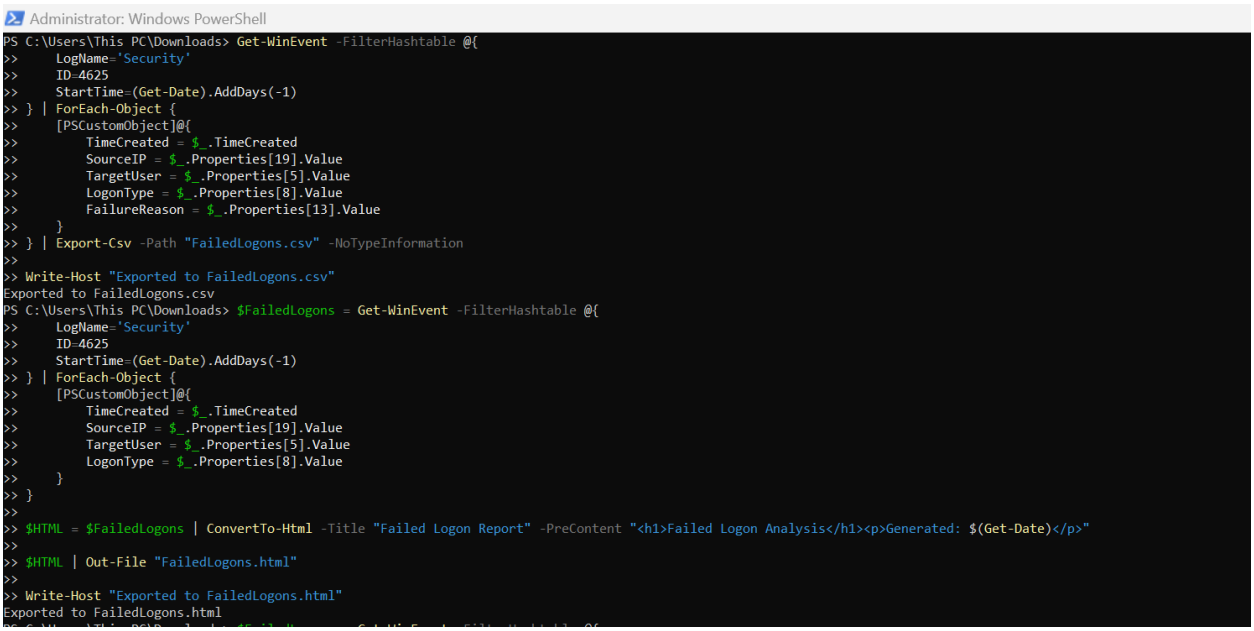
```
        TimeCreated = $_.TimeCreated
        SourceIP = $_.Properties[19].Value
        TargetUser = $_.Properties[5].Value
        LogonType = $_.Properties[8].Value
    }
}

$HTML = $FailedLogons | ConvertTo-Html -Title "Failed Logon Report" -PreContent "<h1>Failed Logon Analysis</h1><p>Generated: $(Get-Date)</p>"

$HTML | Out-File "FailedLogons.html"

Write-Host "Exported to FailedLogons.html"
```

Output:



Observation:

The script generated a clean, readable HTML-based visual report named FailedLogons.html containing a structured summary of failed logon attempts over the last 24 hours. This format is ideal for sharing security review updates with management or archiving incident logs.